INVENTRA

How the INVENTRA Landing Page Was Built

A technical walkthrough of the stack, the files, the rocket-launch animation system, and the desktop/mobile responsiveness approach behind the public marketing page at `/`.

Project: INVENTRA — Inventory & Parcel Tracking | Scope: frontend/src (React + Vite)

1. Overview

The landing page is the public marketing route at `/`, served by `LandingPage.tsx`. It is a single, tall scrolling page built from one fixed navigation bar and six stacked sections (hero, product, about, support, privacy policy, terms, footer). There is no server-rendered content and no CMS — every word, color, and layout rule lives directly in the React/TypeScript component tree and is styled with Tailwind CSS utility classes, with Framer Motion layered on top purely for motion.

Signed-in users never see it: if `useAuth()` reports an active session, the page immediately redirects to `/home` instead of rendering the marketing content, so the route doubles as the public entry point and the authenticated-app gate.

2. Technologies & Tools Used

Category	Tool	Role on the landing page
Framework	React 19 + TypeScript	Component tree, typed props/state for every section.
Build tool	Vite 8	Dev server + production bundling; powers route-level code-splitting.
Routing	React Router v7	`/` route, `Link` navigation to /login, auth-based `Navigate` redirect.
Styling	Tailwind CSS v4	All layout, spacing, color and responsive classes; no separate CSS files per component.
Animation	Framer Motion v12	Every animated effect: launch timeline, scroll parallax, viewport reveals, nav scroll-fade.
Icons	lucide-react	Rocket, ArrowRight, Menu/X, Phone and feature icons (Boxes, Truck, BarChart3, etc.).
Fonts	Google Fonts (Sora, Playfair Display)	Sora = body/UI text; Playfair Display = every serif headline (Tailwind's `font-serif`).

Category	Tool	Role on the landing page
Class utilities	clsx + tailwind-merge (`cn()`)	Conditionally combine/override Tailwind classes without duplication conflicts.
Auth	Supabase Auth (via AuthContext)	Decides whether the landing page renders or redirects to /home.
Type checking	TypeScript (`tsc -b`)	Compile-time safety for every prop and motion-value type.

3. Files Created & Modified

The hero originally lived as one large file with the nav bar and page sections inline. It was later split apart so each concern owns its own file: the hero keeps only the rocket/dashboard animation, and the nav, page sections, shared color tokens, and scroll-reset behavior each moved into their own module.

New files

File	Purpose
frontend/src/components/landing/LandingNav.tsx	Fixed top navigation bar shared by the whole page (not just the hero); scroll-linked glass background, desktop links, mobile hamburger menu.
frontend/src/components/landing/LandingSections.tsx	Product, About, Support, Privacy Policy, Terms sections plus the site footer; shared `reveal()` scroll-in-view animation helper and `Eyebrow` label component.
frontend/src/components/landing/theme.ts	Single source of truth for the landing page's color palette, easing curve, and fixed-nav offset — imported by every landing component.
frontend/src/components/shared/GhostIllustration.tsx	Reusable floating illustration (skeleton-hero.png with a gentle bob animation); shared between the landing page context and the Login page.
frontend/src/app/ScrollToTop.tsx	Router-level helper that resets scroll position to the top on every route change.

Modified files

File	What changed
frontend/src/pages/LandingPage.tsx	Page shell: composes LandingNav + CinematicHero + every section in order; holds the authenticated-user redirect.
frontend/src/components/landing/CinematicHero.tsx	Reworked to a full-viewport hero: single shared 'launch' timeline driving the rocket, headline, and dashboard preview, plus scroll parallax and an idle float.
frontend/src/App.tsx	Registers `<ScrollToTop />` once at the app root; lazy-loads `LandingPage` as its own bundle chunk.

File	What changed
frontend/src/index.css	Global `overflow-x: hidden` guard on html/body, smooth-scroll behavior, base Sora font stack.
frontend/public/skeleton-hero.png	Illustration asset used by GhostIllustration on the hero side-panel context and the login screen.
frontend/index.html	Non-blocking Google Fonts `<link>` for Sora + Playfair Display, page title/meta.

`tailwind.config.ts` was not touched by this feature but is what makes it work: it already mapped Tailwind's `font-serif` utility to Playfair Display and `font-sans` to Sora, so every landing headline just uses `className="font-serif"` to pick up the display font.

---

4. Page Composition

`LandingPage.tsx` is a thin composition root — it renders nothing itself beyond a full-height background wrapper and stacks these pieces top to bottom:

```
<LandingNav />                fixed header, all breakpoints
<CinematicHero />             #hero   — full-viewport intro
<ProductSection />            #product
<AboutSection />              #about
<SupportSection />            #support
<PrivacyPolicySection />      #privacy-policy
<TermsSection />              #terms
<SiteFooter />
```

Product / About / Support / Privacy / Terms are plain in-page anchors (`id="product"` etc.), not separate routes — the nav bar and footer both link to them with ordinary `<a href="#id">` tags, so navigating between them is a same-page scroll, never a route change or a full re-mount.

---

5. Design System (theme.ts)

Every landing component imports its colors from one file instead of hardcoding hex values, so the whole page reads as one consistent brand and a palette change is a single-file edit.

Token	Value	Used for
BLUE	#1677FF	Primary brand color — CTAs, links, icon accents.
CYAN	#4EBBFF	Secondary accent — gradients, the rising orb, chart bars.
NAVY	#10233F	Headline and body text color.
MUTED	#60748D	Secondary/supporting text.
PAGE_BG	#EAF6FF	Base page background.
HERO_BG	#DCEEFF	Hero section background (slightly deeper blue than the page).
CARD_BG	rgba(255,255,255,0.72)	Translucent glass cards over the tinted backgrounds.

Token	Value	Used for
BORDER	rgba(22,119,255,0.14)	Hairline borders on cards, nav, and dividers.
EASE_OUT	cubic-bezier(0.16,1,0.3,1)	The one easing curve used by every animation on the page.
NAV_OFFSET	96 (px)	Reserved height for the fixed nav; mirrors the `scroll-mt-24` Tailwind class on each section.

6. Animation System — How the Rocket & Page Movement Work

All motion on the page comes from Framer Motion, used in four distinct patterns. None of it relies on scroll-jacking (hijacking the browser's native scroll) — an earlier pinned-scroll version of the hero was tried and dropped because it added a visible load delay; the current version animates once on mount and then reacts passively to normal scrolling.

6.1 One shared timeline drives the whole launch

The hero's rocket-and-liftoff effect is not several separately-timed animations — it is a single Framer Motion value called `launch`, animated once from 0 to 1 over 2.4 seconds (with a 0.15s delay) using the imperative `animate()` call inside a `useEffect`, so it can be started, cancelled on unmount, or skipped entirely for reduced-motion users:

```
const launch = useMotionValue(reduced ? 1 : 0);
useEffect(() => {
  if (reduced) { launch.set(1); return; }
  launch.set(0);
  const controls = animate(launch, 1, { duration: 2.4, delay: 0.15, ease: EASE_OUT });
  return () => controls.stop();
}, [reduced, launch]);
```

Every moving piece of the hero is a separate `useTransform` that reads from that same `launch` value, each mapped to its own output range. Because they all share one 0→1 clock but travel different distances and finish at different points along it, the layers appear to move at different speeds without any per-element delay or a second timer:

Layer	Interpolates over	Effect
Rocket position	launch 0→1 → y: 420px → -560px	Travels the full visible height, off-screen bottom to off-screen top — the fastest, most dramatic motion.
Rocket opacity	launch 0→0.1→0.75→1 → opacity 0→1→1→0	Fades in almost instantly, stays visible through the flight, then fades out before the timeline ends — it exits rather than parking on screen.
Hero copy (badge/headline/CTAs)	launch 0→1 → y: 44→0, opacity finishes by 0.4	Smallest travel distance; settles well before the rocket disappears, reading as 'left behind' by it.
Background glow / rising orb	launch 0→1 → opacity 0→0.55 by 0.5	Slowest-finishing layer, gives the launch a lingering atmospheric backdrop.

Layer	Interpolates over	Effect
Dashboard preview mock	launch 0→1 → y: 130→0, opacity finishes by 0.45	Rises into place just after the copy, completing the composition.

All five rows read from the exact same launch value — only the output range and the opacity keyframe stops differ per layer.

6.2 Scroll-linked parallax (after the launch settles)

Independently of the launch timeline, `useScroll({ target: heroRef, offset: ["start start", "end start"] })` tracks how far the visitor has scrolled through the hero itself, producing a 0→1 `scrollYProgress`. That drives two more `useTransform` outputs — the orb drifts up to 48px, the dashboard up to 18px — which are added on top of the settled launch position (`useTransform([a, b], ([a, b]) => a + b)`). The orb moving faster than the dashboard as you scroll past the hero is the classic parallax depth cue, and it is entirely separate from the one-time rocket animation.

6.3 Idle float on the dashboard preview

Once settled, the dashboard preview gets a small perpetual bob (`y: [0, -6, 0]`, 6-second loop) via `useAnimationControls()`. It only runs while the hero is on screen (`useInView`) *and* the browser tab is active (a `visibilitychange` listener) — it is explicitly stopped otherwise, so it costs nothing once the visitor scrolls past the hero or switches tabs.

6.4 Section reveal-on-scroll

Every section below the hero (feature cards, About, Support, the legal sections) fades up and unblurs into place the first time it enters the viewport, via a shared `reveal()` helper in `LandingSections.tsx`:

```
initial:     { opacity: 0, y: 28, filter: 'blur(6px)' }
whileInView: { opacity: 1, y: 0,  filter: 'blur(0px)' }
viewport:    { once: true, margin: '-80px' }
transition:  { duration: 0.7, ease: EASE_OUT }
```

`whileInView` is backed by an IntersectionObserver, not a scroll listener, so it costs nothing while a section is off-screen and animates exactly once (`once: true`). The four product-capability cards pass an increasing delay (`0.05 * index`) into the same helper to stagger in left to right.

6.5 Nav bar scroll transition

The fixed nav starts fully transparent (blending into the hero) and fades to a soft glass bar as the page scrolls. This is done with `useTransform(scrollY, [0, 90], [...])` feeding straight into the inline `style` of a `motion.div` — it never touches React state or triggers a re-render while scrolling, Framer Motion writes the interpolated background/border/shadow colors directly to the DOM.

6.6 Reduced motion

Every animated component calls Framer Motion's `useReducedMotion()` (reading the OS-level accessibility preference). When it's on: the launch value jumps straight to its resting state with no animate() call, the rocket's opacity keyframes evaluate to 0 for its entire (invisible) existence, the idle float and section reveal animations are skipped outright, and the nav's mount transition renders with no initial offset. Nothing on the page requires motion to be understood.

---

7. Desktop / Mobile Responsiveness

There is one component tree for every screen size — no separate mobile layout or JS resize/media-query logic. Responsiveness is entirely Tailwind's mobile-first breakpoint classes (unprefixed = mobile baseline, then `sm:` / `md:` / `lg:` layer on top).

- **Fluid type, not stepped sizes** — the hero H1 and every section heading use CSS `clamp()` directly in the className (e.g. `text-[clamp(2.1rem,5.4vw,4.25rem)]`), so headline size scales continuously with viewport width instead of jumping at each breakpoint.
- **Nav** — full inline links + Login/Get Started buttons at `md:` and above; below that it collapses to a single hamburger button that toggles a dropdown panel built from local `useState`.
- **Hero CTAs** — the two buttons stack full-width in a column on mobile (`flex-col`) and sit side by side at `sm:flex-row`.
- **Dashboard preview** — its width narrows in three steps (94% of its container on mobile, 82% at `sm`, 62% at `md`+), and its internal stat-tile grid goes from 2 columns to 4 at `sm:grid-cols-4`.
- **Product cards** — 1 column on mobile, 2 at `sm:grid-cols-2`, 4 at `lg:grid-cols-4`.
- **Cost-cutting on mobile** — the decorative grain-noise texture layer in the hero is `hidden` below `sm:`; it's a purely decorative desktop touch, not rendered (and not paid for) on small screens.
- **Spacing scale** — section padding steps up per breakpoint (`py-20 sm:py-28`, etc.) rather than using one fixed value everywhere.
- **Horizontal overflow guard** — `overflow-x-hidden` is set on the page root and globally on `html`/`body` in index.css, so full-bleed absolutely-positioned decoration (the rising orb, the grain texture) can never introduce a horizontal scrollbar on narrow viewports.

---

8. Routing, Navigation & Performance Details

- **Lazy-loaded route** — `LandingPage` is registered with `React.lazy()` and a shared `<Suspense>` fallback in `App.tsx`, so its code (and its Framer Motion usage) ships as its own bundle chunk — a visit straight to `/dashboard` never downloads it.
- **Scroll reset on navigation** — React Router doesn't reset scroll position the way a full page load does. `ScrollToTop.tsx` is a headless component (`return null`) mounted once at the app root that runs `window.scrollTo(0, 0)` in a `useLayoutEffect` on every pathname change — without it, clicking 'Get Started' partway down the landing page would land on /login at that same scroll offset.
- **Anchor sections, not routes** — Product/About/Support/Privacy/Terms are same-page anchors with `scroll-mt-24`, so a fixed-nav click scrolls to the section without it disappearing under the bar, and without a route transition or component remount.
- **Auth-aware redirect** — if `useAuth()` reports `isAuthReady && isAuthenticated`, the component returns `<Navigate to="/home" replace />` instead of the marketing markup, so a signed-in visitor to `/` is bounced straight into the app.

---

9. Summary

The landing page is a plain React/TypeScript component tree styled with Tailwind and animated with Framer Motion — no custom animation engine, no scroll-jacking, no separate mobile build. The 'rocket carrying the page

upward' effect is one shared 0→1 motion value read by five differently-scaled transforms; everything after that (parallax, idle float, section reveals, the nav fade) is driven by ordinary scroll position or viewport-intersection, gated behind `useReducedMotion()` throughout. Responsiveness is handled entirely by Tailwind's breakpoint classes and CSS `clamp()` on one shared layout, with no device-specific code paths.

Generated as project documentation for INVENTRA. Reflects the codebase under `frontend/src` at the time of writing.